ACTIVIDAD 8

Simulación de incidentes y ejecución de runbooks



Vértice AI Consulting · Convertimos datos en decisiones

Materia:	*Gestión de Proyectos de Inteligencia Artificial*
Actividad:	Actividad 8 · Módulo 3 / Semana 10 · Temas 19 y 20
Alumno(a):	*Eduardo Aburto Angulo*
Matrícula:	*AL07182743*
Docente:	*M. en C. Luis Ariel Vázquez Piña*
Fecha de entrega:	5 de noviembre de 2026

Declaración de Uso de Inteligencia Artificial

El alumno debe indicar de manera honesta si utilizó herramientas de Inteligencia Artificial (IA) para el desarrollo de esta actividad. Completa la siguiente tabla:

Herramienta de IA utilizada	¿La usaste?	Propósito / Descripción de uso
ChatGPT / OpenAI	Sí	ChatGPT/Codex: apoyo en programación, ejecución de pruebas, simulaciones locales, documentación y revisión visual. Las mediciones proceden de ejecuciones reales con fallas inyectadas.
Claude (Anthropic)	Sí	Revisión técnica independiente de la entrega: clonado del repositorio, ejecución propia de scripts/verify_evidence.py —que recalcula las 900 solicitudes, las 15 ventanas, las métricas por ventana, la correlación de trazas y los pasos de runbook— y de las 9 pruebas automatizadas; verificación del SHA256 del modelo contra el manifiesto y la Actividad 6; cotejo de los entregables contra los cuatro criterios de la rúbrica y los apartados exigidos; y elaboración de las instrucciones de corrección. No redactó el código ni los documentos.
Gemini (Google)	Antecedentes	Herramienta mencionada en actividades previas; no se atribuye ejecución nueva en Actividad 8.
Copilot (Microsoft)	Antecedentes	Herramienta mencionada en actividades previas; no se atribuye ejecución nueva en Actividad 8.
Otra (especifica):	Antecedentes	Herramienta mencionada en actividades previas; no se atribuye ejecución nueva en Actividad 8.

Desarrollo de Contenido

Diseño de la prueba

Preparé tres fallas reversibles dentro del servicio local, conservando intactos los pesos y el umbral del modelo. Las reglas y acciones se implementaron antes de ejecutar el laboratorio; este documento describe la ejecución y sus resultados. Utilicé un canario de 60 noticias históricas con etiquetas conocidas y solicitudes secuenciales.

Fase	Ventanas	Intervención
Normal	0 a 2	Referencia sin falla
Latencia	3 a 6	Retraso de 350 ms; corrección tras ventana 4
Calidad	7 a 10	Probabilidad 1 − p; corrección tras ventana 8
Drift	11 a 14	Texto repetido ocho veces; corrección tras ventana 12

La detección requiere dos ventanas consecutivas malas. La resolución requiere dos ventanas sanas. El hash SHA-256 del modelo se comprobó antes y después: f48c7496d5a12bd15d535721e3e83c6ef895b1ae7eaccbe7f4f4c60705bb1916. Las claves temporales se mantienen fuera de las evidencias.

Incidente de latencia

Inyecté 350 ms adicionales por predicción. El p95 máximo de esta fase fue 364.14 ms. RB01 detectó burn rate excesivo, comprobó alcance y restauró mode=normal. La detección tomó 43.457 s; el cierre tras dos ventanas sanas tomó 0.291 s desde la detección. Las 120 solicitudes lentas permanecen en el presupuesto histórico.

Acción preventiva: medir bajo concurrencia y definir ventanas temporales antes de producción. La prueba es de respuesta a retraso inyectado; no identifica un cuello de botella real de infraestructura.

Incidente de calidad

Invertí la probabilidad servida mediante 1 − p después de la inferencia. Así simulo una degradación de salida sin sustituir ni reentrenar el modelo. RB02 detectó la caída de recall o F1 con 20 etiquetas negativas por ventana, diagnosticó el modo de servicio y restauró la configuración normal.

Estado	Recall	F1
Antes	0.9500	0.9268
Durante	0.4500	0.2609
Después	0.9500	0.9268

La detección tomó 0.295 s y la recuperación verificada 0.268 s. No se atribuye la recuperación a entrenamiento ni a promoción de un modelo nuevo. Acción preventiva: añadir validación contractual de salida y mantener alerta abierta cuando no existan etiquetas suficientes.

Incidente de drift

Repetí ocho veces los textos del lote para desplazar su distribución de longitud. RB03 detectó PSI mayor a 0.20, puso en cuarentena el flujo sintético y volvió a usar la entrada de referencia. El escalamiento quedó como ticket local con reentrenamiento no ejecutado.

El PSI máximo fue 7.572; volvió a 0.000. La detección tomó 0.298 s y la recuperación verificada 0.261 s. La prueba demuestra drift de longitud; no identifica un cambio conceptual ni exige por sí sola reentrenar.

Acción preventiva: observar otras variables, comprobar origen y calidad del lote y solicitar evaluación humana antes de actualizar el modelo. Una cuarentena productiva requeriría almacenamiento separado, permisos y política de recuperación; aquí se controla el generador local.

Alertas y presupuesto de error (error budget)

El presupuesto de error (error budget) es el número de fallas que tolera el SLO dentro del conjunto observado: N × (1 − SLO). Distingo lo permitido, lo consumido y el saldo; aquí N=900 solicitudes, no un mes de servicio.

SLO	Permitido	Consumido	Saldo
Disponibilidad 99 %	9	0	9
Latencia 99 % ≤250 ms	9	120	-111

El SLO de latencia no se cumplió. La configuración se recuperó y las últimas dos ventanas fueron sanas, pero recuperar el servicio no restituye el presupuesto gastado. El burn rate divide la fracción de fallas entre 0.01: es un multiplicador del ritmo tolerado. Elegí >2.0 para detectar un consumo superior al doble de ese ritmo; es una decisión de laboratorio, no un valor óptimo demostrado. Exijo dos ventanas consecutivas de 60 solicitudes para reducir aperturas por fluctuaciones aisladas.

La persistencia y deduplicación redujeron seis avisos crudos a tres notificaciones emitidas: 50 % menos. Esta comparación mide volumen de avisos, no una tasa de falsos positivos. Priorizar y verificar la recuperación protege la continuidad sin ocultar el incumplimiento histórico.

Cronología y cierre

Caso	Inyección UTC	Detección UTC	Resolución UTC
INC-001	00:40:51.197	00:41:34.654	00:41:34.945
INC-002	00:41:34.949	00:41:35.244	00:41:35.512
INC-003	00:41:35.516	00:41:35.813	00:41:36.074

Fecha UTC de la tabla: 6 de noviembre de 2026; fecha local de entrega: 5 de noviembre. MTTD medio 14.683 s; MTTR medio 0.273 s. La corrección exacta y las seis etapas de cada runbook están en evidencias/runbook_steps.json; incidentes completos en evidencias/incidents.json.

Resultados y decisiones

Los tres incidentes finalizaron resolved y el modelo conservó su hash. La corrida demuestra el ciclo de detectar, intervenir y verificar. La disponibilidad fue 100 %, mientras el SLO agregado de latencia quedó incumplido. No oculto ese resultado: es el efecto esperado de la falla deliberada y una prueba de que el presupuesto registra el daño incluso después de la recuperación.

Mantengo como límites la escala de un host, muestra repetida y etiquetas inmediatas. En un servicio real las etiquetas tardarían en llegar y las acciones necesitarían control de concurrencia, persistencia y guardias. Mi recomendación es validar esas condiciones antes de permitir que las señales apoyen decisiones operativas de clientes.

Cómo auditar

Revisar la configuración en config/alertas.json, los procedimientos en runbooks/README.md y las mediciones originales. Ejecutar python scripts/verify_evidence.py recalcula métricas y comprueba alertas, trazas, hash y recuperación. No es necesario confiar en las cifras impresas del reporte para contrastar el resultado.

Tiempos por incidente y exposición antes de detectar

Caso	Detección s	Recuperación s	Solicitudes lentas antes de detectar
INC-001	43.457	0.291	120
INC-002	0.295	0.268	0
INC-003	0.298	0.261	0

Resumen	Detección s	Recuperación s
Media	14.683	0.273
Mediana	0.298	0.268
Máximo	43.457	0.291

Por incidente reporto tiempo de detección y recuperación; MTTD y MTTR son sus medias entre incidentes. La mediana describe mejor el centro de estos tres ensayos, mientras el máximo revela la exposición más prolongada. Ninguno permite estimar tiempos productivos con una muestra tan pequeña.

La asimetría se explica por solicitudes secuenciales: dos ventanas de 60 con 350 ms adicionales acumulan al menos 42 s de demora inyectada, más inferencia y medición. Calidad y drift también esperan dos ventanas; terminan antes porque no añaden esa espera por solicitud. No son alertas instantáneas.

Antes de detectar INC-001 ya habían finalizado 120 solicitudes lentas. Son 60 textos repetidos dos veces, no 120 clientes. Como banquero, ese volumen de respuestas demoradas es el costo operativo que el promedio por sí solo oculta. El cálculo usa los timestamps originales entre inyección y detección; scripts/analyze_incidents.py deja resultados y hashes de entrada en evidencias/incident_analysis.json.

Propongo evaluar ventanas más cortas para latencia y un disparador rápido junto al persistente. No lo implementé ni atribuyo mejoras medidas: anticipar avisos puede reducir exposición, a costa de más ruido y mayor carga para el operador. Requeriría comparar políticas con tráfico representativo antes de cambiar la configuración.

Conclusiones

La respuesta funcionó en los tres ensayos, pero INC-001 expuso 120 solicitudes lentas antes del aviso. Con ánimo de reducir esa exposición, propongo comparar políticas de detección antes de cambiarlas. Como banquero Sr., priorizo continuidad y confianza del cliente sobre un promedio favorable que oculte el peor caso.

La IA no sustituye el criterio del banquero; lo aumenta.

Fundamento de las decisiones

Relaciono indicadores y respuesta con los temas del curso (Universidad Tecmilenio, 2026). El presupuesto y la propuesta de ventanas rápidas se apoyan en Google (2018), sin atribuir al laboratorio la implementación de ventanas temporales productivas. Registro parámetros, métricas y artefactos conforme al propósito de Tracking (MLflow, s. f.).

La API utiliza FastAPI (s. f.) y el refresco del tablero usa fragmentos de Streamlit (s. f.). Las métricas de clasificación siguen las definiciones de scikit-learn (s. f.); el corpus financiero se atribuye a Malo et al. (2014). OpenTelemetry (s. f.) aporta el marco conceptual de spans y relaciones padre-hijo: aquí conservé instrumentación local propia, sin desplegar su plataforma distribuida.

Referencias

FastAPI. (s. f.). FastAPI. Recuperado el 6 de noviembre de 2026, de https://fastapi.tiangolo.com/

Google. (2018). Alerting on SLOs. En The site reliability workbook. https://sre.google/workbook/alerting-on-slos/

Malo, P., Sinha, A., Korhonen, P., Wallenius, J., & Takala, P. (2014). Good debt or bad debt: Detecting semantic orientations in economic texts. Journal of the Association for Information Science and Technology, 65(4), 782–796. https://doi.org/10.1002/asi.23062

MLflow. (s. f.). MLflow Tracking. Recuperado el 6 de noviembre de 2026, de https://mlflow.org/docs/latest/ml/tracking/

OpenTelemetry. (s. f.). Traces. Recuperado el 6 de noviembre de 2026, de https://opentelemetry.io/docs/concepts/signals/traces/

scikit-learn. (s. f.). Metrics and scoring: Quantifying the quality of predictions. Recuperado el 6 de noviembre de 2026, de https://scikit-learn.org/stable/modules/model_evaluation.html

Streamlit. (s. f.). st.fragment. Recuperado el 6 de noviembre de 2026, de https://docs.streamlit.io/develop/api-reference/execution-flow/st.fragment

Universidad Tecmilenio. (2026). Gestión de proyectos de inteligencia artificial: Temas 19 y 20 y rúbrica de la Actividad 8 [Material del curso].

Cobertura de la rúbrica

Criterio (25 % cada uno)	Apartado de este documento	Página PDF/Word
Diseño de monitorización y observabilidad	Diseño de la prueba	3
Diseño y aplicación de alertas inteligentes	Alertas y presupuesto de error (error budget)	5
Simulación de incidentes y uso de runbooks	Incidente de calidad	4
Documentación de la respuesta automatizada	Resultados y decisiones	6

Las páginas corresponden a la edición PDF y Word verificada. En Google Docs la paginación puede variar; los encabezados conservan la localización del contenido.

Este documento fue estructurado precisamente para cubrir esos 4 apartados de forma explícita y verificable.